

Automation with AI and MCP Servers using Playwright-Notes-2

- **Installation-** Github copilot with Playwright MCP Server
Install Github Copilot extension from VS Code- Install MCP server using the following commands:

pip install mcp =>this is for Python

installs mcp with command line tools included

pip install "mcp[cli]"

check mcp is installed

pip show mcp

(or)

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": [
        "@playwright/mcp@latest"
      ]
    }
  }
}
```

Add this manually in your settings.json file in VS Code which can be accessed by going to **settings in the bottom left corner in VS Code=>Click on Open Settings (JSON) option**

(or)

Go to this website: <https://github.com/microsoft/playwright-mcp> and click on **Install Server VS Code**

(or)

npm install -g @playwright/mcp@latest and npm list -g @playwright/mcp@latest - type this command in the terminal/cmd

Tested Github Copilot working along with Playwright MCP server using the basic prompts:

1. Go to this website <https://wishinfinite.com/>, click on Playground and then click on Selectors under Playground using Playwright MCP Server

2.Go to www.amazon.com and click on Best Sellers section using Playwright MCP Server

- **Perform UI and Functional Automation Testing using Github Copilot and Playwright MCP Server-**

In Github Copilot type the following prompts:

- 1. go to www.wishinfinite.com, then navigate to the E-commerce App section under the Playground tab. Then fill in the username and password in the Login form and submit the form and confirm successful login using the Playwright MCP Server**
- 2. go to <https://www.saucedemo.com/> and fill in the username and password and submit the form and confirm successful login using Playwright MCP Server**
- 3. go to www.wishinfinite.com, then navigate to the E-commerce App section under the Playground tab. Then fill in the username and password in the Login form and submit the form and confirm successful login. Order any 2 items and under view orders validate that the correct items got ordered. Once all the steps are done log out from the application. Perform all these steps using the Playwright MCP Server**
- 4. go to <https://www.saucedemo.com/> and fill in the username and password and submit the form and confirm successful login. Order any 2 items and go to the cart and validate that the correct items got ordered. Once all the steps are done log out from the application. Perform all these steps using the Playwright MCP Server**

Note: whatever prompt you want to feed into Github Copilot, refine the prompt using ChatGPT or Claude first

No code BDD Testing with Playwright and Github Copilot and Cucumber.js-

- **BDD**- Behaviour Driven Development- way of writing tests in plain English that everyone — developers, testers, and even non-technical people like managers — can read and understand.

Simple analogy:

Normal tests are like cooking instructions written in code — only chefs (developers) can read them.

BDD tests are like a recipe written in plain English — anyone can read and understand exactly what the dish should taste like.

BDD uses Gherkin language:

BDD tests are written in a special plain English format called **Gherkin** using three keywords:

GIVEN → the starting situation / setup

WHEN → the action that happens

THEN → the expected result

Real world example:

gherkin

Feature: Login functionality

Scenario: Successful login with valid credentials

Given I am on the saucedemo login page

When I enter username "standard_user"

And I enter password "secret_sauce"

And I click the login button

Then I should be redirected to the products page

And I should see the page title "Products"

Anyone — developer, manager, tester — can read this and understand exactly what is being tested.

BDD structure explained:

Feature → what big feature are we testing?

"Login functionality"

Scenario → one specific situation to test

"Successful login with valid credentials"

Given → setup / starting condition

"I am on the login page"

When → action performed

"I enter username and password"

And → additional step (used with Given/When/Then)

"And I click login"

Then → expected outcome

"I should see products page"

Multiple scenarios in one feature:

gherkin

Feature: Login functionality

Scenario: Successful login

Given I am on the login page

When I enter valid credentials

Then I should see the products page

Scenario: Login with wrong password

Given I am on the login page

When I enter wrong password

Then I should see an error message

Scenario: Login with empty fields

Given I am on the login page

When I click login without entering anything

Then I should see a validation error

BDD in Playwright

Playwright supports BDD through a library called Cucumber combined with Playwright.

How it works:

Step 1 → Write feature file in plain English (Gherkin)

↓
Step 2 → Write step definitions in TypeScript
(connects plain English to actual Playwright code)
↓
Step 3 → Run tests
↓
Step 4 → Cucumber reads feature file,
matches each step to Playwright code,
executes the browser actions

Installation:

bash

install cucumber with playwright

npm install @cucumber/cucumber

npm install @playwright/test

install ts-node for TypeScript support

npm install ts-node

install types

npm install @types/node

Project structure:

project/

```
|
|— features/
|   |— login.feature    ← plain English test
|
|— steps/
|   |— loginSteps.ts    ← Playwright code for each step
|
|— cucumber.json        ← cucumber config
|— playwright.config.ts  ← playwright config
```

Step 1 — Write the feature file:

features/login.feature

gherkin

Feature: Login functionality

Scenario: Successful login with valid credentials

Given I am on the saucedemo login page
When I enter username "standard_user"
And I enter password "secret_sauce"
And I click the login button
Then I should be redirected to the products page

Step 2 — Write step definitions:

steps/loginSteps.ts

typescript

```
import { Given, When, Then } from "@cucumber/cucumber"
import { Browser, Page, chromium } from "@playwright/test"
```

```
let browser: Browser
```

```
let page: Page
```

```
// GIVEN — setup step
```

```
Given("I am on the saucedemo login page", async () => {
  // launch browser and navigate to page
  browser = await chromium.launch({ headless: false })
  const context = await browser.newContext()
  page = await context.newPage()
  await page.goto("https://www.saucedemo.com/")
})
```

```
// WHEN — action steps
```

```
When("I enter username {string}", async (username: string) => {
  // fill username field
  await page.locator("#user-name").fill(username)
})
```

```
When("I enter password {string}", async (password: string) => {
  // fill password field
  await page.locator("#password").fill(password)
})
```

```
When("I click the login button", async () => {
  // click login button
  await page.locator("#login-button").click()
})
```

```
// THEN — assertion step
Then("I should be redirected to the products page", async () => {
  // verify URL changed to inventory page
  await page.waitForURL("**/inventory.html")
  console.log("Successfully redirected to products page")
  await browser.close()
})
```

Step 3 — Configure cucumber:

cucumber.json

```
json
{
  "default": {
    "require": ["steps/**/*.ts"],
    "requireModule": ["ts-node/register"],
    "format": ["progress", "html:reports/report.html"]
  }
}
```

Step 4 — Run BDD tests:

bash

run all feature files

npx cucumber-js

run specific feature file

npx cucumber-js features/login.feature

run with tags

npx cucumber-js --tags "@smoke"

Using tags to organize scenarios:

gherkin

Feature: Login functionality

@smoke @login

Scenario: Successful login

Given I am on the login page

When I enter valid credentials

Then I should see products page

@regression

Scenario: Failed login

Given I am on the login page

When I enter wrong credentials

Then I should see error message

bash

run only smoke tests

npx cucumber-js --tags "@smoke"

run only regression tests

npx cucumber-js --tags "@regression"

BDD vs Normal Playwright tests:

	Normal Playwright		BDD Playwright	
Written in	TypeScript/JavaScript		Plain TypeScript	English +
Who can read	Developers only		Everyone	
File type	.spec.ts		.feature + .ts	
Format	Code		Gherkin + Code	
Extra library needed	✗ No		✓ Cucumber	
Best for	Developer testing	focused	Team collaboration	wide

Full flow summary:

Business team says:

"Users should be able to login with valid credentials"

↓

Written as Gherkin in .feature file:

"Given I am on login page

When I enter valid credentials

Then I see products page"

↓

Developer writes step definitions:

Given → page.goto(url)

When → page.fill(username) + page.fill(password)

Then → `page.waitForURL(productsPage)`



Cucumber runs feature file
matches each line to step definition
Playwright executes browser actions



Test passes or fails
Report generated

Bottom line:

BDD is writing tests in plain English so everyone on the team understands what is being tested — not just developers.

BDD in Playwright combines Cucumber (plain English test runner) with Playwright (browser automation) — Cucumber reads your plain English scenarios and Playwright executes the actual browser actions behind each step.

@smoke and **@regression** — Layman's Terms

These are tags used to label and group your tests so you can choose which ones to run at any given time.

Simple analogy:

Think of your tests like a playlist of songs.

- **@smoke** is a "quick favourites" playlist — just the most important songs
- **@regression** is the "full album" playlist — every single song

You choose which playlist to play depending on how much time you have.

@smoke

Smoke tests are a small set of the most critical tests that verify the basic functionality of your app is working.

The name comes from hardware testing — engineers would turn on a device and check if smoke came out. If no smoke → good enough to test further.

gherkin

@smoke

Scenario: Successful login

Given I am on the login page

When I enter valid credentials

Then I should see the products page

@smoke

Scenario: Page loads correctly

Given I navigate to the home page

Then the page title should be visible

When to run smoke tests:

- ✓ After every code deployment
- ✓ Before running full test suite
- ✓ When you have very little time
- ✓ Quick sanity check — "is the app alive?"
- ✓ First thing in the morning

Like checking if your car starts and moves before taking it on a long trip — you are not testing everything, just the basics.

@regression

Regression tests are a complete set of all tests that verify nothing that was working before has been broken by new code changes.

The word "regression" means going backwards — regression testing makes sure the app has not regressed (gone back) to a broken state.

gherkin

@regression

Scenario: Login with wrong password

Given I am on the login page

When I enter wrong password

Then I should see an error message

@regression

Scenario: Login with empty fields

Given I am on the login page

When I click login without entering anything

Then I should see a validation error

@regression

Scenario: Add product to cart

Given I am logged in

When I click add to cart

Then the cart count should increase

When to run regression tests:

- ✓ Before a major release
- ✓ After a big code change
- ✓ End of day or overnight runs

✓ Weekly full test runs

✓ Before merging a big feature branch

Like a full car service before a long road trip — checking everything thoroughly.

Side by side comparison:

@smoke

@regression

Small set of tests

Complete set of ALL tests

Most critical only

Every scenario

Runs in minutes

Runs in hours

Quick sanity check

Full verification

Run after every deploy

Run before releases

"Is the app alive?"

"Is everything still working?"

Using both tags together:

gherkin

Feature: Login functionality

@smoke @regression

Scenario: Successful login

Given I am on the login page

When I enter valid credentials

Then I see the products page

included in BOTH smoke and regression runs

@regression

Scenario: Login with wrong password

Given I am on the login page

When I enter wrong password

Then I see an error message

only in regression run

@regression

Scenario: Login with empty fields

Given I am on the login page

When I click login with empty fields

Then I see a validation error

only in regression run

@smoke @regression

Scenario: Products page loads

Given I am logged in

Then I see the products list

included in BOTH smoke and regression runs

Running specific tags:

bash

run only smoke tests — fast, quick check

npx cucumber-js --tags "@smoke"

run only regression tests — full suite

`npx cucumber-js --tags "@regression"`

run tests that are BOTH smoke AND regression

`npx cucumber-js --tags "@smoke and @regression"`

run tests that are smoke OR regression

`npx cucumber-js --tags "@smoke or @regression"`

run everything EXCEPT smoke

`npx cucumber-js --tags "not @smoke"`

Other common tags you will see:

gherkin

@sanity → even smaller than smoke — absolute basics only

@critical → must never fail — highest priority

@flaky → known unstable tests — run separately

@slow → tests that take long — run overnight

@login → all tests related to login feature

@cart → all tests related to cart feature

@wip → work in progress — not ready yet

Real world workflow:

Developer pushes code



Run @smoke tests (2 minutes)

"Is the basic app working?"



If smoke passes → merge the code



End of day → Run @regression tests (1 hour)

"Did anything break anywhere?"



Before release → Run @regression again

"Is everything ready to ship?"

Bottom line:

T
a
g

Think
of it
as

When to
run

@
s
m
o
k
e

Quick
healt
h
chec
k

After
every
deploym
ent

@
r
e

Full
body
chec
kup

Before
every
release

g
r
e
s
s
i
o
n

@smoke answers — "Is the app alive?" @regression answers — "Is everything still working correctly?"

In VSCode:

1. Install **Cucumber (Given When Then)** extension in VSCode
2. Create a new folder called **features**
3. In the features folder create all your .features files-refer to the **Automation-with-Playwright-MCP-and AI folder** in VS Code
4. After creating the .features files test your scenarios in your features files using Github Copilot or claude with the following prompt:

Take my feature files and run all the scenarios with Playwright MCP Server, performing the browser actions described in all the scenarios. Note: I want to directly execute test scenarios in chrome browser using Playwright MCP server and do not generate script

- **API Testing with Playwright MCP and Github Copilot-**
 1. Install **executeAutomation server** in Playwright using the below command:
npm install -g @executeautomation/playwright-mcp-server
 2. Verify that the above package has been installed using the below command:
npm list -g @executeautomation/playwright-mcp-server

- **API-** Application Programming Interface. It is a way for two different applications to talk to each other-acts as an intermediary between 2 systems, allowing them to exchange data and functionality
-

Simple analogy:

Think of a restaurant:

- You are the customer
- The kitchen is the system that has the data/functionality
- The waiter is the API

You don't walk into the kitchen yourself and cook your food. You tell the waiter what you want. The waiter takes your request to the kitchen, gets the food, and brings it back to you.

The API is the waiter — it carries requests and responses between two systems.

Real world example:

When you use a weather app on your phone:

Weather App on your phone



sends request to Weather API

"give me weather for Mumbai"



Weather API talks to weather database

fetches the data



sends back response to your app

"Mumbai: 32°C, Sunny"



Your app displays it

Your app did not store the weather data itself — it asked the API and got it back

How an API works:

Step 1 → You make a REQUEST to the API

"Give me user details for id 1"



Step 2 → API processes the request
goes to database, finds the data



Step 3 → API sends back a RESPONSE
{ name: "Alice", email: "alice@email.com" }

API Request and Response:

REQUEST contains:

- URL/endpoint "https://api.example.com/users/1"
- Method GET, POST, PUT, DELETE
- Headers authentication, content type
- Body data you are sending (for POST/PUT)

RESPONSE contains:

- Status code 200 success, 404 not found, 500 error
 - Headers content type, server info
 - Body the actual data returned as JSON
-

HTTP Methods explained simply:

GET → fetch/read data
"give me user details"

POST → create new data
"create a new user"

PUT → update existing data completely
"update all details of this user"

PATCH → update existing data partially
"only update the email of this user"

DELETE → delete data

"delete this user"

What is JSON:

APIs usually send and receive data in **JSON format** — a simple structured text format.

```
json
{
  "id": 1,
  "name": "Alice",
  "email": "alice@email.com",
  "age": 25
}
```

JSON is just a **structured way of writing data** — like a very organized list that both machines and humans can read.

Status codes — what the API tells you back:

200 → OK — request successful
201 → Created — new data was created successfully
400 → Bad Request — you sent something wrong
401 → Unauthorized — you need to login first
403 → Forbidden — you don't have permission
404 → Not Found — the data doesn't exist
500 → Internal Server Error — something broke on the server

Real world APIs you use every day:

Google Maps API → shows maps in apps
PayPal API → handles payments
Twilio API → sends SMS messages
Weather API → provides weather data
Anthropic API → powers Claude AI
Spotify API → provides music data
Twitter API → posts and reads tweets

API vs Website:

	Website	API
Made for	Humans to see	Applications to use
Returns	HTML pages	JSON data
Opened in	Browser	Code or tools like Postman
Has visuals	 Yes	 No — just data

Bottom line:

Concept	Simple meaning
API	Waiter between two applications
Request	What you ask for
Response	What you get back
JSON	Format the data comes in
Status code	Whether it worked or not

An API is simply a **messenger** — you send a request, it goes and gets what you need, and brings it back. You never need to know how the kitchen works — you just talk to the waiter.

HTTP Methods and CRUD Operations — Layman's Terms

Simple analogy:

Think of a **library**:

- You can **read** a book
- You can **add** a new book
- You can **replace** an old book with a new edition

- You can **remove** a book

CRUD and HTTP methods are just these same four basic operations applied to data.

CRUD Operations

CRUD stands for the **4 basic operations** you can perform on any data:

C → Create add new data
R → Read fetch/get existing data
U → Update modify existing data
D → Delete remove data

HTTP Methods

HTTP methods are the **actual commands** used in APIs to perform CRUD operations:

POST → Create
GET → Read
PUT → Update (complete)
PATCH → Update (partial)
DELETE → Delete

How CRUD maps to HTTP Methods:

CRUD Operation	HTTP Method	What it does
----------------	-------------	--------------

Create	→ POST	create new data
Read	→ GET	fetch existing data
Update	→ PUT / PATCH	modify existing data
Delete	→ DELETE	remove data

Status codes for each method:

GET → 200 OK
POST → 201 Created
PUT → 200 OK
PATCH → 200 OK
DELETE → 204 No Content

Complete summary:

CRUD	HTTP	Status	What happens
------	------	--------	--------------

Create	→ POST	→ 201	→ new data created
Read	→ GET	→ 200	→ data returned
Update	→ PUT	→ 200	→ all data replaced
Update	→ PATCH	→ 200	→ partial data updated
Delete	→ DELETE	→ 204	→ data removed

Bottom line:

CRUD	HTTP	Library analogy
Create	POST	Adding a new book
Read	GET	Reading a book
Update	PUT	Replacing entire book
Update	PATCH	Fixing one chapter
Delete	DELETE	Removing a book

CRUD are the **4 things you can do with data**. HTTP methods are the **actual commands** you use to do them in an API.

Website used for practice:

<https://fakerestapi.azurewebsites.net/index.html>

In VSCode type the following prompts into Github Copilot:

1. Perform GET API call for request URL <https://fakerestapi.azurewebsites.net/index.html> and verify that we are getting status code as 200. Also generate GET call response body. Use Playwright MCP server to perform this API call

2. Perform API call for the curl command `curl -X GET "https://fakerestapi.azurewebsites.net/api/v1/Activities/20" -H "accept: text/plain; v=1.0"` and verify that we are getting status code as 200. Also generate GET call response body. Use PlaywrightEA MCP server to perform this API call

3. Perform POST API call for request URL <https://fakerestapi.azurewebsites.net/index.html>. Here we can use the below request body:

```
{
  "id": 99,
  "title": "MCP Server Learning",
  "dueDate": "2026-04-30T22:55:50.047Z",
  "completed": true
}
```

and verify that we are getting status code as 200. Also validate that the response body is matching the below request body:

```
{
  "id": 99,
  "title": "MCP Server Learning",
  "dueDate": "2026-04-30T22:55:50.047Z",
  "completed": true
}
```

Use PlaywrightEA MCP server to perform this API call

4. Perform DELETE API call for request URL <https://fakerestapi.azurewebsites.net/index.html> and validate that response status code is 201. Also generate GET call response body. Use PlaywrightEA MCP server to perform this API call. Use PlaywrightEA MCP server to perform this API call